

ELUCIDATING INFERENCE IN MULTI-TOKEN PREDICTION: PROPOSAL, VERIFICATION, AND PARALLELIZATION

Anonymous authors

Paper under double-blind review

ABSTRACT

Multi-token prediction promises substantial inference speedups by emitting several tokens per forward pass, but the quality of the resulting text depends critically on how those tokens are accepted, rejected, or resampled. We study this decoding layer directly, treating the multi-token model as a proposal mechanism and asking what acceptance rule best trades throughput against generation quality and alignment to an underlying autoregressive (AR) reference. We organize the design space along three axes — how proposals are *verified*, how many and how long the proposals are, and how proposal and verification are *parallelized* — and we give a generative model of the number of correct tokens per call that turns the second axis into an exactly solvable allocation problem. Our experiments are conducted in PTP, a recent multi-token inference framework, but the acceptance rules and the evaluation protocol are framework-agnostic and apply to any model producing multiple tokens per call. We report accepted-tokens-per-call, wall-clock speedup, and downstream task quality across 62 decoding configurations on Spec-Bench, and find that the three measures induce three different orderings: committing more tokens per call is neither sufficient for lower latency nor compatible with staying faithful to the reference.

1 INTRODUCTION

Autoregressive decoding emits one token per forward pass. A multi-token model breaks that constraint by proposing several tokens at once, but a proposal is not an output: something must decide which of the proposed tokens are kept. That decision — the *acceptance rule* — is usually treated as a fixed implementation detail, inherited from speculative decoding (Leviathan et al., 2023; Chen et al., 2023), where it is chosen to leave the base model’s output distribution exactly intact. Once the proposal mechanism is strong enough to offer many coherent tokens per call, that choice stops being incidental. It sets the throughput, and it sets how far the generated text drifts from what the base model would have produced.

This paper takes the decoding layer as the object of study. We ask the question a practitioner actually faces: *on a single GPU, what is the lowest latency we can achieve that still produces high-quality outputs?* We organize the answer along three axes.

Verification. What counts as a correct token? The strictest rule accepts a proposed token only if the base model, driven by the same randomness, would have emitted exactly that token. Relaxing it to “the base model finds this token plausible” — nucleus membership, a probability threshold, a likelihood ratio — accepts far more tokens per call, at a cost in faithfulness that we measure rather than assume.

Proposal geometry. How many chances to take, and how long each should be. Given a node budget, is it better to propose one long continuation, or fifty short ones? We answer this by modelling the number of correct tokens per call as the extinction time of a branching process, which makes the expected yield separable across depth and the allocation problem exactly solvable.

Parallelization. Verification and the next proposal can run concurrently if one speculates on the verification outcome. Partial Quadratic Decoding (PQD) (Draxler et al., 2025) does this by preparing

a branch for each possible number of correct tokens and spending a budget across those branches. We show that the branch weights and the value of a branch can be read off the same calibrated model that solves the geometry problem, rather than estimated separately.

We make three main contributions.

- **The acceptance layer, made explicit.** We give a taxonomy of seven families of acceptance rules for multi-token proposals and implement them behind a single code path, so that two configurations differ only in the acceptance predicate. Alongside it we give a measurement protocol that separates throughput from faithfulness, including a test of whether the generated text is distributionally distinguishable from the base model’s (§3).
- **A generative model of N_{cor} that makes proposal geometry solvable.** A zero-inflated Beta-Binomial branching process with shared per-generation frailty predicts per-call accepted-token histograms across three orders of magnitude of proposal width from three parameters ($R^2 \geq 0.976$). Its survival function makes expected yield separable across depth, which turns strand-length selection into a knapsack we solve exactly by dynamic programming — and the same object supplies the branch weights and branch values that Partial Quadratic Decoding needs, replacing two separately calibrated estimates with one (§4–§5.1).
- **Evidence that the standard metric is the wrong one alone.** Across 62 decoding configurations on Spec-Bench, accepted-tokens-per-call, wall-clock speedup, and faithfulness to the base model induce three different orderings. The configurations at the top of the tokens-per-call ranking get there by accepting almost everything, roughly halving the base model’s mean token probability; among those that stay close to the base model, more tokens per call often means *slower* generation — wide ensembles reach 4.6 tokens per call at $0.31 \times$ autoregressive speed. Only the exact rules are distributionally indistinguishable from the base model (§6).

2 BACKGROUND: MULTI-TOKEN PROPOSALS AND THEIR VERIFICATION

2.1 AUXILIARY VARIABLES

We build on Parallel Token Prediction (PTP) (Draxler et al., 2025), which we recap only as far as needed. A standard decoder predicts $P_i := P(t_i \mid t_{<i})$ and then samples $t_i \sim P_i$. The sampling step is where the sequential dependency lives, and it can be made explicit: drawing $u_i \sim \mathcal{U}[0, 1]$ and taking the inverse CDF,

$$t_i = \text{Pick}(u_i, P_i) \equiv \min \left\{ j : \sum_{l \leq j} P_{il} > u_i \right\}, \quad (1)$$

makes t_i a *deterministic* function of the context and the auxiliary variable u_i . Since u_i determines t_i given $t_{<i}$, it carries the same information as the token itself, so a model given $u_i, \dots, u_k$ as inputs can predict t_k without knowing the intervening tokens. This is the basis of PTP: a model $P_\theta(t_k \mid t_{<i}, u_i, \dots, u_k)$ emits many tokens in one call. Figure 1 illustrates the shift of randomness from post-hoc sampling to a random model input. We use the one-hot variant (O-PTP), whose output probability for the proposed token doubles as a confidence estimate c_k .

2.2 WHAT COUNTS AS A CORRECT TOKEN

Sharing the auxiliary variables gives verification a strong coupling. A proposed token t_k is *correct* if it equals $\tilde{t}_k = \text{Pick}(u_k, Q_\phi(\cdot \mid t_{<k}))$, the token the base model selects when the *same* auxiliary is applied to its own distribution. Let

$$N_{\text{cor}} = \max \{ m \geq 0 : t_{i+j} = \tilde{t}_{i+j} \text{ for all } j < m \} \quad (2)$$

be the longest correct prefix. Because the base model evaluates $Q_\phi(\cdot \mid t_{<k})$ at every position during verification anyway, one token can be sampled from it for free at the first rejection, so a call commits

$$N_{\text{acc}} = N_{\text{cor}} + 1 \quad (3)$$

tokens. Under this rule the committed text is distributed exactly as the base model’s, so speedup is free of quality cost; the only question is how large N_{cor} is.

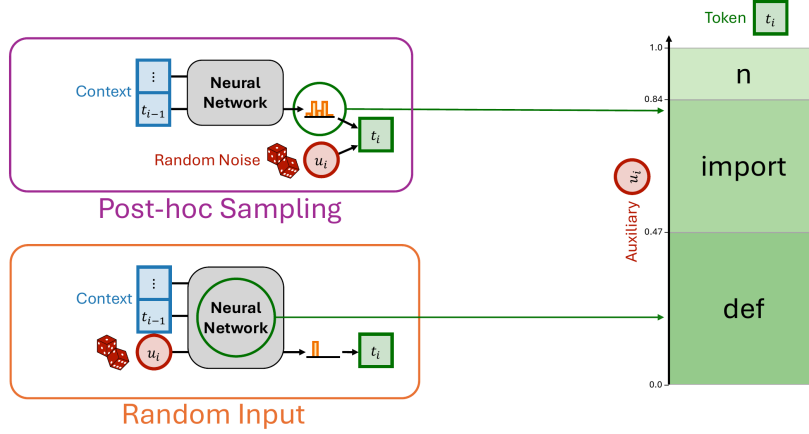


Figure 1: Moving the randomness. **Top:** standard decoding draws a distribution, then samples from it with auxiliary u_i . **Bottom:** PTP feeds u_i in as an input, so the token becomes a deterministic function of context and auxiliaries and future tokens become jointly predictable. **Right:** u_i selects a token by inverse CDF. Reproduced from the PTP framework (Draxler et al., 2025).

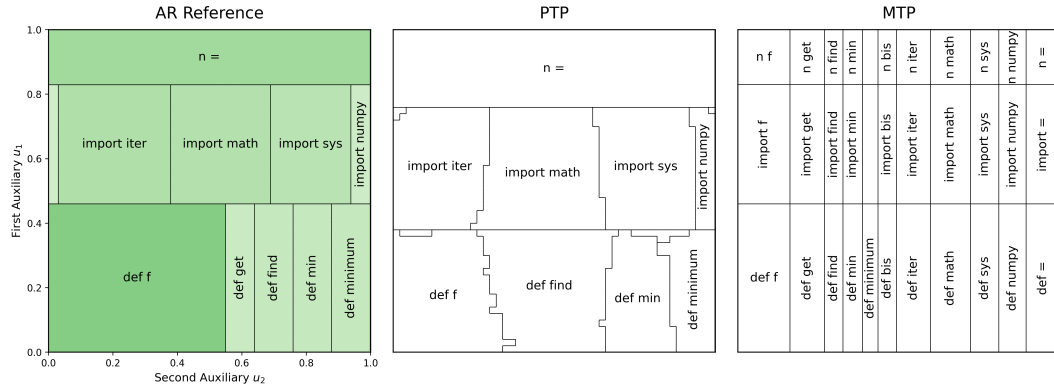


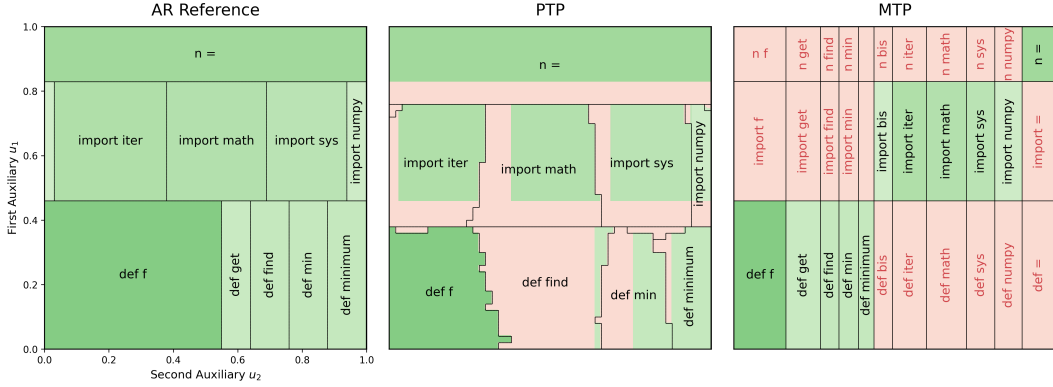
Figure 2: Verification as a partition-matching problem, for the first two generated tokens. The AR reference (left) partitions the auxiliary square into one cell per token pair. PTP (middle) reproduces the curved cell boundaries; independent multi-token prediction (right) can only form a product partition of rectangles. Correctness of a proposal is agreement of the two partitions at the drawn (u_1, u_2) . Reproduced from Draxler et al. (2025).

Figure 2 makes the geometry concrete for two tokens. The auxiliary pair (u_1, u_2) ranges over the unit square, and the AR reference partitions that square into regions, one per token pair. A multi-token model induces its own partition; where the two agree, the proposal is correct. PTP’s partition follows the reference’s curved cell boundaries closely, whereas independent per-position prediction can only produce a product partition of axis-aligned rectangles and therefore mismatches over a large area.

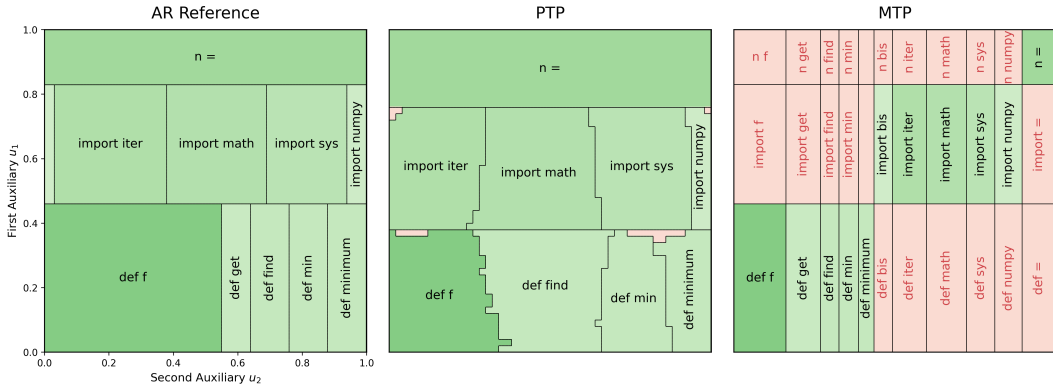
3 THE ACCEPTANCE LAYER

3.1 EXACT ACCEPTANCE IS STRICT

The rule of §2.2 is exact but unforgiving: a proposal is rejected whenever the drawn auxiliary falls in the sliver where the two partitions disagree, even if the proposed token is one the base model would have been perfectly happy to emit at a slightly different u . On the example of Figure 3, exact acceptance keeps 63% of the auxiliary square; relaxing the criterion to nucleus membership keeps 99% of it, with the same proposal model and the same proposals.



(a) Exact acceptance: keep the proposal only where it *equals* the reference token at the same auxiliary.



(b) Broadened acceptance: keep the proposal wherever the reference finds the token plausible (nucleus membership).

Figure 3: The acceptance rule, not the model, decides what is kept. Both rows show the *same* proposal partitions as Figure 2; green is accepted, pink rejected. Broadening the criterion converts most of the rejected area into accepted area. This is the single largest lever in the design space, and the one that trades faithfulness for throughput most directly.

3.2 A TAXONOMY OF RULES

Write q for the proposal model’s distribution at a position, p for the base model’s, x for the proposed token, and u for the shared auxiliary. Every rule we study decides acceptance from these objects; the families differ in what they look at. All are implemented in a single code path so that the only difference between two runs is the predicate.

Exact. Accept iff $x = \text{Pick}(u, p)$. Distribution-preserving by construction (§2.2).

Relaxed deterministic. Accept iff x lies in a high-probability set of p : nucleus membership, $\sum_{y:p(y)>p(x)} p(y) < \theta$ (Holtzman et al., 2020), or top- k rank. As $\theta \rightarrow 0$ this contracts to argmax agreement, not to the exact rule.

Thresholds. Accept iff $p(x) \geq \theta$ (base-model probability), or iff $q(x) \geq \theta$ (the proposal model’s own confidence). The latter needs no base-model call at all, which makes it the cheapest rule available and, as we show, not a good one.

Stochastic ratio. The speculative-sampling rule (Leviathan et al., 2023; Chen et al., 2023): accept with probability $\min(1, (\kappa p(x) + \varpi)/q(x))$ and, on rejection, resample from the residual $\propto \max(0, p - q)$. At $\kappa = 1, \varpi = 0$ this is distribution-preserving in theory.

Inverse-CDF distance. Accept iff the drawn u is within θ of the interval $[u_{\text{lo}}, u_{\text{hi}}]$ that p assigns to x , i.e. $\max(u_{\text{lo}} - u, u - u_{\text{hi}}) \leq \theta$. This interpolates continuously from the exact rule at $\theta = 0$, and

is the natural relaxation in the auxiliary-square picture of Figure 3: it dilates the accepted cell rather than replacing the criterion.

Unconditional. Accept the first k proposed tokens regardless. A throughput ceiling and a quality floor.

Self-verification. Replace the base model by the proposal model’s own autoregressive prediction, so no second set of weights is consulted. This removes the base-model call from the critical path entirely; the cost is that the rule no longer references the distribution we are trying to stay faithful to.

Only the exact rule and (in theory) the ratio rule preserve the base model’s output distribution. Every other rule biases the output, and the empirical question is by how much, in exchange for what.

4 A MODEL OF THE NUMBER OF CORRECT TOKENS

The verification axis fixes *when* a token is kept. The geometry axis asks how many proposals to make and how long each should be, and answering it requires knowing how N_{cor} behaves. We therefore assume a model of N_{cor} given the number of proposal strands K and the context, and we will need from it only the survival function

$$S[w, d] = \Pr[T \geq d \mid w \text{ strands}], \quad T = N_{\text{acc}} - 1, \quad (4)$$

where T is the number of generations before every strand has failed verification and, following equation 3, $G := N_{\text{acc}} = 1 + T$ is the committed token count per call.

4.1 GEOMETRIC AND BETA-GEOMETRIC BASELINES

The simplest model treats each accepted token as an independent coin flip with success probability p , making T geometric:

$$S[1, d] = p^{d-1}, \quad \Pr[T = d] = (1 - p)p^{d-1}. \quad (5)$$

For a single strand this is already a good description: Figure 4 fits equation 5 to 5,070 observed calls at $K = 1$ with $R^2 = 0.999$. Allowing the acceptance probability to vary across prompts, $p \sim \text{Beta}(a, b)$, gives the beta-geometric model

$$\Pr[T = d] = \int_0^1 (1 - p)p^{d-1} \text{Beta}(p; a, b) dp = \frac{B(a + d - 1, b + 1)}{B(a, b)}, \quad (6)$$

which matters because that variation is real: a likelihood-ratio test against a single pooled p rejects homogeneity decisively ($\chi^2 = 401$ on 99 degrees of freedom, $p \approx 5 \times 10^{-38}$; Figure 11).

4.2 ENSEMBLES NEED A CORRELATED MODEL

Neither baseline survives the move to $K > 1$ strands. If strands failed independently, total wipeout would vanish geometrically in K ; empirically $\Pr[T = 1]$ falls from 41% at $K = 1$ only to 13% at $K = 1000$ and flattens against a floor (Figure 6, left). Some positions are simply hard, and no ensemble width rescues them.

We therefore model the strands as a branching process with a frailty that is *shared* within each generation. At depth i , given $S_{i-1} = m$ surviving strands,

$$\Theta_i \sim \pi_0 \delta_0 + (1 - \pi_0) \text{Beta}(\alpha, \beta), \quad S_i \mid S_{i-1} = m, \Theta_i \sim \text{Binomial}(m, \Theta_i), \quad (7)$$

with $T = \min\{i \geq 1 : S_i = 0\}$. We parameterize by the marginal per-strand success probability p and the within-generation correlation ρ between two strands,

$$\nu = \frac{1 - \rho}{\rho}, \quad \alpha = p\nu, \quad \beta = (1 - p)\nu. \quad (8)$$

Two features are load-bearing. The shared draw of Θ_i correlates strands within a generation: they tend to fail together, which is why width has diminishing returns. The point mass π_0 at $\Theta = 0$ makes

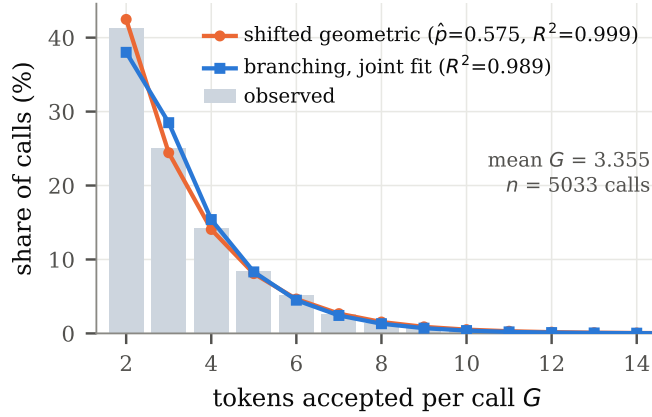


Figure 4: Committed tokens per call at $K = 1$. A shifted geometric equation 5 fitted to this width alone is already excellent; the branching model of §4.2, fitted *jointly* across all five widths, gives up a little accuracy here in exchange for extrapolating across K (Figure 6). Preliminary single-seed run.

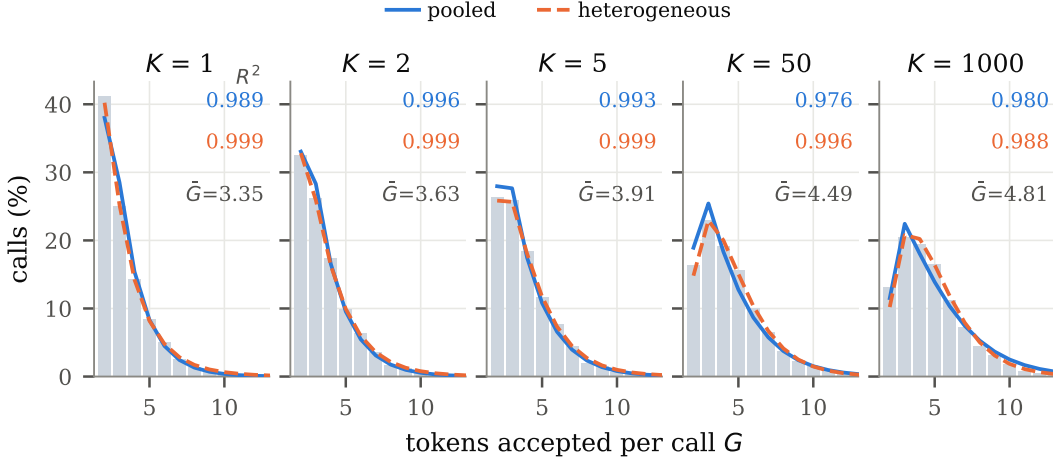


Figure 5: Committed tokens per call against ensemble width. Bars are observed; lines are the branching model equation 7 with a single (π_0, p, ρ) shared across all five panels (*pooled*) and with p integrated over a fitted $\text{Beta}(a, b)$ population (*heterogeneous*). R^2 per curve is printed in the curve’s own colour. One three-parameter model tracks the distribution over three orders of magnitude of K .

a fraction of positions a guaranteed wipeout regardless of K , producing the observed floor; a pure Beta-Binomial cannot. Marginalizing Θ_i gives a Beta-Binomial transition kernel

$$P[m, j] = (1 - \pi_0) \text{BetaBin}(j \mid m, \alpha, \beta) + \pi_0 \mathbb{I}[j = 0], \quad (9)$$

with 0 absorbing, from which $S[w, \cdot]$ follows by propagation. Following the structure of the decoder, generation 1 is exempt from zero-inflation and the terminal generation $d_{\max}$ is a forced death, reflecting the hard proposal-length cap. Fitting (π_0, p, ρ) by joint maximum likelihood over all five widths simultaneously gives $\pi_0 = 0.128$, $p = 0.620$, $\rho = 0.789$ on our checkpoint.

5 FROM SURVIVAL TO ALLOCATION: THE STRAND-LENGTH KNAPSACK

Strands need not all be the same length. Let L_s be the length budgeted to strand s and define the *width profile*

$$W(d) := \#\{s : L_s \geq d\}, \quad (10)$$

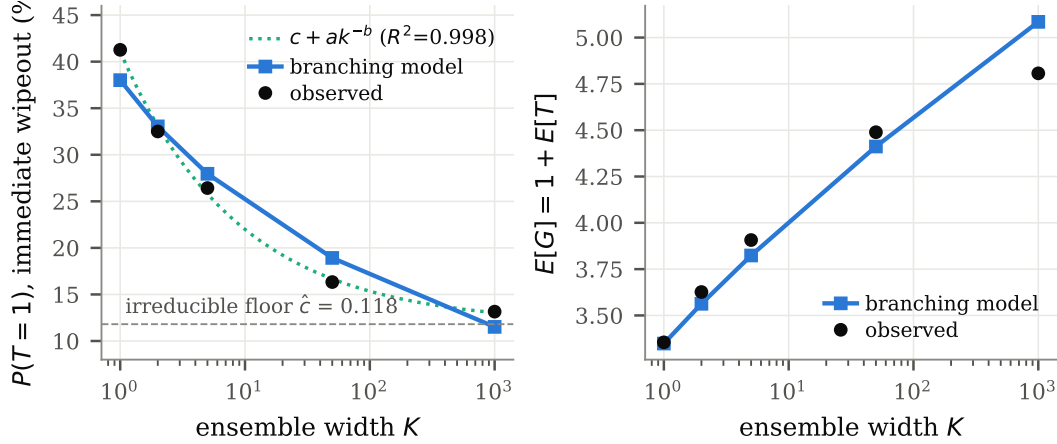


Figure 6: Why width saturates. **Left:** the probability of immediate wipeout decays towards an irreducible floor $\hat{c} = 0.118$, not to zero; a phenomenological $c + aK^{-b}$ captures this, and the branching model produces it from π_0 . **Right:** committed tokens per call grow roughly logarithmically in K — a $1000\times$ increase in proposal width buys about 1.5 extra tokens.

the number of strands still budgeted to reach depth d . The key structural fact is that the shared frailty Θ_d acts conditionally i.i.d. on every strand present in generation d , so any fixed subset of a surviving population is itself Beta-Binomial with the same (α, β) — a thinning property. Consequently the subpopulation that is both alive and still within its own budget at depth d is distributed exactly as a fresh homogeneous run started with $W(d)$ strands, *independently of every other depth*, and the tail-sum identity gives

$$\mathbb{E}[T] = \sum_{d \geq 1} \Pr[T \geq d] = \sum_{d=1}^{d_{\max}} S[W(d), d]. \quad (11)$$

Expected yield is a sum of terms each depending on the width profile at a *single* depth. That separability is what makes the design problem tractable.

Choosing strand lengths under a node budget is then

$$\max_{L_1, \dots, L_K} \sum_{d=1}^{d_{\max}} S[W(d), d] \quad \text{s.t.} \quad \sum_s L_s \leq C, \quad (12)$$

a knapsack whose items are *whole strands*: buying a strand of length L raises the width at every depth $1, \dots, L$ at once, so its cost is L and its value is the bundled gain across those depths.

Exact solution. The feasible width profiles are exactly the nonincreasing integer sequences $W(1) \geq \dots \geq W(d_{\max}) \geq 0$: any such sequence decomposes into $W(d) - W(d+1)$ strands of length d , by the standard correspondence between nonincreasing sequences and partitions, and its cost $\sum_d W(d)$ equals the sum of those lengths. So equation 12 is a maximization over nonincreasing profiles, solved exactly by a dynamic program over depth:

$$V_d[w_{\text{cap}}, b] = \max_{0 \leq w \leq \min(w_{\text{cap}}, b)} \left\{ S[w, d] + V_{d+1}[w, b - w] \right\}, \quad V_{d_{\max}+1} \equiv 0, \quad (13)$$

where $V_d[w_{\text{cap}}, b]$ is the best achievable $\sum_{d' \geq d} S[W(d'), d']$ given $W(d) \leq w_{\text{cap}}$ and remaining budget b . Doubling the width bound until the optimum stops touching it makes the result exact at any budget.

5.1 COMBINING WITH PARTIAL QUADRATIC DECODING

Speculative decoding runs proposal and verification in sequence. PQD removes that serialization by speculating on the verification outcome: while the base model verifies N_{prop} proposed tokens,

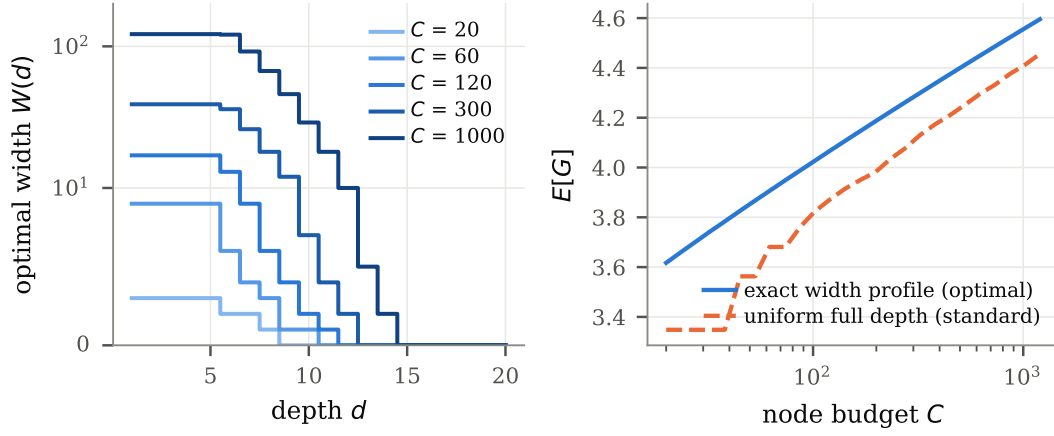


Figure 7: The allocation solved by equation 13. **Left:** optimal width profiles at several node budgets — wide and shallow, never uniform. **Right:** what the exact profile buys over spending the same budget on full-depth strands (the *standard* allocation). The gap is roughly 0.15–0.2 committed tokens per call across two decades of budget.

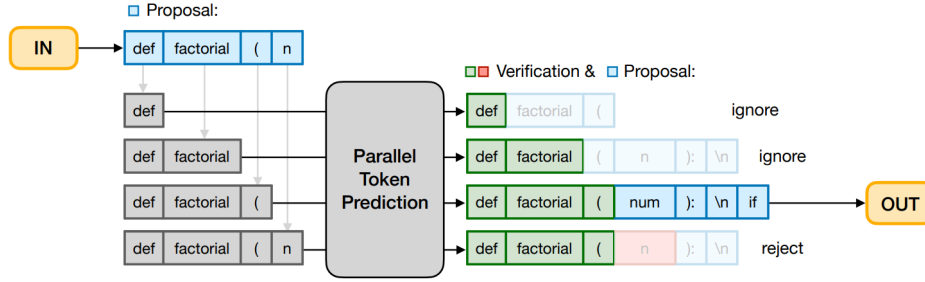


Figure 8: Partial Quadratic Decoding. One branch is prepared per possible number of correct tokens, concurrently with verification; when the base model returns m^* , the matching branch is already extended and the rest are discarded. Reproduced from Draxler et al. (2025).

the proposal model prepares a branch for each possible $m \in \{0, \dots, N_{\text{prop}}\}$, each conditioned on “exactly m of the proposed tokens were correct” (Figure 8). When verification returns m^* , the other branches are discarded and generation continues from the precomputed branch m^* .

PQD must decide how many continuation tokens B_m to give each branch under a total budget C , which it does by maximizing an expected utility

$$\max_{\sum_m B_m = C} \sum_m \Pr[N_{\text{cor}} = m \mid t] H(B_m), \quad (14)$$

solved greedily by marginal gain. As originally formulated, the two ingredients of equation 14 come from different places: the branch weights are estimated by treating errors as independent given the proposal model’s confidences, $\Pr[N_{\text{cor}} = m \mid t] \approx (1 - c_{i+m}) \prod_{k < i+m} c_k$, while the reward H is a separate monotone function calibrated on the training set.

§4 supplies both from one object. The branch weights are the increments of the survival function,

$$\Pr[N_{\text{cor}} = m] = S[K, m] - S[K, m + 1], \quad (15)$$

and the value of giving a branch n tokens is the expected yield of the optimal allocation of those n tokens, i.e. the dynamic program equation 13:

$$H(n) = \max_{W \text{ nonincreasing, } \sum_d W(d) \leq n} \sum_d S[W(d), d] = V_1[\infty, n]. \quad (16)$$

Both ingredients are then consistent by construction, calibrated by the same three parameters, and — because H is the *optimal* yield rather than a heuristic proxy — the greedy allocation in equation 14

Table 1: Fitted survival function $S[w, d] = \Pr[T \geq d]$ for a homogeneous run of w strands, at $(\pi_0, p, \rho) = (0.128, 0.620, 0.789)$. Width has no value at depth 1 and sharply diminishing value thereafter.

w	depth d						
	1	2	3	4	5	6	8
1	1.000	0.620	0.335	0.181	0.098	0.053	0.015
2	1.000	0.670	0.386	0.221	0.125	0.071	0.022
5	1.000	0.720	0.444	0.269	0.161	0.096	0.033
50	1.000	0.811	0.557	0.373	0.245	0.159	0.064

Table 2: Optimal versus uniform allocation of a node budget, under the fitted model. The profile column reads $\text{depth}^{\text{count}}$, so 6^{29} means 29 strands of length 6. The exact profile trades depth for width and gains ≈ 0.15 committed tokens per call.

budget C	allocation	profile $\text{depth}^{\text{count}}$	strands	$E[G]$
120	exact profile	$11^1 10^1 9^2 8^2 7^3 6^4 5^4$	17	4.067
	uniform full depth	20^6	6	3.873
300	exact profile	$12^2 11^2 10^3 9^5 8^6 7^8 6^{10} 5^3$	39	4.282
	uniform full depth	20^{15}	15	4.114
1000	exact profile	$14^2 13^3 12^5 11^8 10^{11} 9^{17} 8^{21} 7^{25} 6^{29} 5^1$	122	4.555
	uniform full depth	20^{50}	50	4.412

is comparing branches on the quantity it actually cares about. The independence approximation is retained only where per-position confidences are genuinely more informative than the pooled model, namely in conditioning S on context; §6.4 compares pooled, Bayesian-updated, and learned-head estimates of p empirically.

5.2 WORKED EXAMPLE

Table 1 gives the fitted survival function at our checkpoint’s $(\pi_0, p, \rho) = (0.128, 0.620, 0.789)$. Two things stand out. $S[w, 1] = 1$ for every $w \geq 1$, so width has *zero* marginal value at depth 1 — a single strand always reaches the first generation. And the returns to width at a fixed depth are strongly concave: at depth 4, going from 1 to 2 strands buys more than going from 5 to 50.

Table 2 turns this into allocations. At a budget of $C = 1000$ nodes — the setting our $K = 50$ runs use — the optimum is 122 strands of mixed length, from 2 of depth 14 down to 29 of depth 6, and it yields 4.555 committed tokens per call against 4.412 for 50 strands at full depth 20. The exact solver reallocates the deep tail of the uniform profile into extra width at shallow depths, where the survival table says it is worth more.

6 EXPERIMENTS

6.1 SETUP

We distil a one-hot PTP student from Vicuna-7B-v1.5 (Zheng et al., 2023) with gated LoRA adapters (Hu et al., 2022) of rank 128 and a binary auxiliary embedding, and evaluate on the first turns of the 13 Spec-Bench categories (Xia et al., 2024) (100 questions), with temperature 0.7, top- k 50, top- p 0.9, at most 20 proposed tokens per strand and a total budget of 120 proposed tokens per call, on a single NVIDIA RTX A6000.

We report three families of measure, deliberately kept separate. *Throughput per call* is N_{acc} averaged over calls. *Wall-clock* is milliseconds per generated token, and speedup is the AR run’s ms/token divided by the rule’s — so it charges every rule for the compute it actually spends. *Faithfulness* is measured under the base model: $\bar{q}$ is the mean per-token probability the base model assigns to the generated tokens (%), PPL the corresponding perplexity, and p_{align} the p -value of a pooled

Table 3: Acceptance rules on Spec-Bench. $\bar{q}$ is the mean per-token probability under the base model, p_{align} the p -value for “generated tokens came from the base distribution”. † distribution-preserving by construction. Numbers are preliminary: single-seed runs on 100 Spec-Bench first turns.

acceptance rule	setting	tok./call	ms/tok.	speedup	$\bar{q}$ (%)	PPL	p_{align}
autoregressive †	–	1.00	56.4	1.00 $\times$	84.15	1.20	0.251
PTP, single call †	–	2.39	32.7	1.73 $\times$	84.51	1.19	0.247
PTP, sequential †	–	3.63	55.3	1.02 $\times$	84.71	1.19	0.799
nucleus	$\theta = 0.9$	3.88	35.4	1.59 $\times$	74.09	1.40	<0.001
nucleus	$\theta = 0.98$	4.30	33.2	1.70 $\times$	67.58	1.56	<0.001
nucleus + ensemble	$\theta = 0.98, K = 5$	4.32	57.4	0.98 $\times$	82.92	1.22	<0.001
nucleus + ensemble	$\theta = 0.98, K = 50$	4.74	140.4	0.40 $\times$	82.97	1.22	<0.001
speculative ratio	–	2.35	34.3	1.65 $\times$	69.94	1.58	<0.001
speculative ratio	$\kappa = 10$	2.62	31.0	1.82 $\times$	70.12	1.57	<0.001
teacher threshold	$\theta = 0.1$	3.58	46.0	1.23 $\times$	83.16	1.21	<0.001
teacher threshold	$\theta = 0.5$	3.22	52.3	1.08 $\times$	85.26	1.18	<0.001
student confidence	$\theta = 0.5$	2.01	69.4	0.81 $\times$	41.42	17.03	<0.001
student confidence	$\theta = 0.9$	1.34	92.7	0.61 $\times$	64.95	1.77	0.006
accept- k	$k = 2$	2.00	31.7	1.78 $\times$	49.29	40.92	<0.001
accept- k	$k = 4$	4.00	16.1	3.50 $\times$	47.34	31.99	<0.001
self-verified	exact	3.86	49.9	1.13 $\times$	65.92	1.60	<0.001
self-verified + nucleus	$\theta = 0.8$	6.45	39.8	1.42 $\times$	24.66	38.70	<0.001
self-verified + ensemble	$\theta = 0.8, K = 50$	5.98	111.1	0.51 $\times$	73.74	1.40	<0.001

CLT z -test of the null hypothesis that the generated tokens were drawn from the base distribution. A large p_{align} is the operational meaning of “indistinguishable from the base model”.

6.2 ACCEPTANCE RULES: THREE ORDERINGS, NOT ONE

Table 3 is the main result. The headline is that the three measures disagree.

Tokens per call is not speed. Sequential exact PTP commits 3.63 tokens per call — more than any single-call rule — yet reaches only 1.02 $\times$, because its proposal and verification passes are serialized. The single-call variant commits 2.39 and reaches 1.73 $\times$. Ensembles push tokens per call to 4.74 and wall-clock *down* to 0.40 $\times$: at $K = 50$ the 1000 proposal nodes cost more than the tokens they save. This is precisely the regime PQD (§5.1) exists to fix, and it is invisible if one reports only accepted-tokens-per-call.

Only the exact rules are faithful. The z -test separates the two classes cleanly: $p_{\text{align}} = 0.25$ and 0.80 for exact PTP, versus < 0.001 for every relaxed rule. Broadening to nucleus $\theta = 0.98$ costs 16.6 points of $\bar{q}$ (84.5 $\rightarrow$ 67.6) to gain 1.9 tokens per call.

Cheap rules are bad rules. Student-confidence thresholding needs no base-model call, and is the worst rule in the table on every axis at once: at $\theta = 0.5$ it commits 2.01 tokens per call, runs *slower* than AR (0.81 $\times$), and collapses quality to PPL = 17.0. Accepting k tokens unconditionally is the honest version of the same trade: 3.50 $\times$ at $k = 4$, with PPL = 32.0.

An anomaly worth flagging. The stochastic ratio rule is distribution-preserving in theory at $\kappa = 1, \varpi = 0$, but we measure $\bar{q} = 69.9$ and $p_{\text{align}} < 0.001$ — a clear deviation. We have not marked it as lossless in Table 3 for that reason. Resolving whether this is an artefact of residual resampling in the truncated top- k space or a genuine implementation defect is left to future work; we report it rather than assume the theory.

6.3 THRESHOLD SWEEPS

Figure 10 sweeps θ for the three threshold-style families. Base-model-probability thresholding degrades gracefully and monotonically: as $\theta \rightarrow 0$ it accepts everything, reaching 17 tokens per call and 4.1 $\times$, and as θ grows it recovers the base model’s quality. Nucleus acceptance occupies a narrow, useful band near $\theta \approx 0.9$ –0.98. Student-confidence thresholding is non-monotone and dominated everywhere.

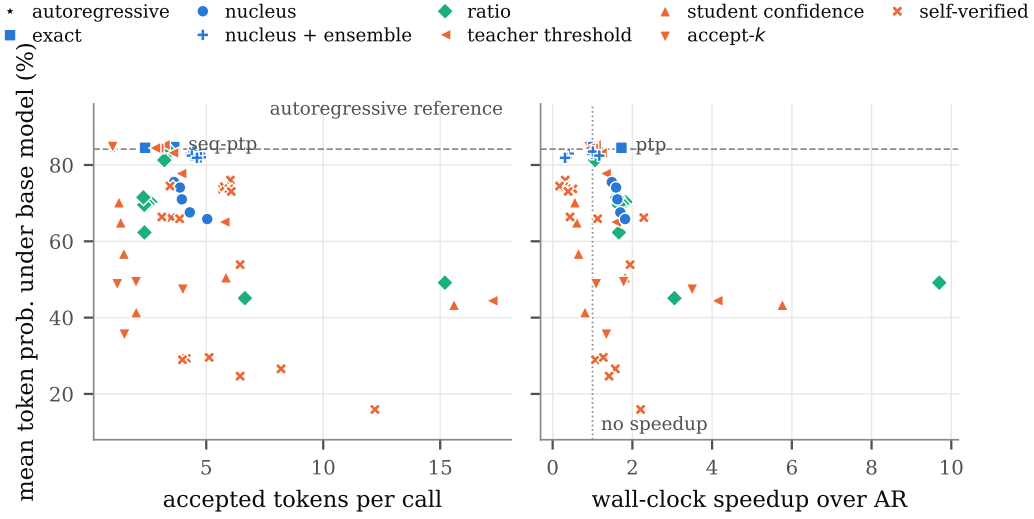


Figure 9: Throughput against faithfulness for all 62 configurations. The two panels share a y -axis but order the rules differently: high tokens per call (left) does not imply wall-clock speedup (right). Only the exact rules sit on the autoregressive reference line.

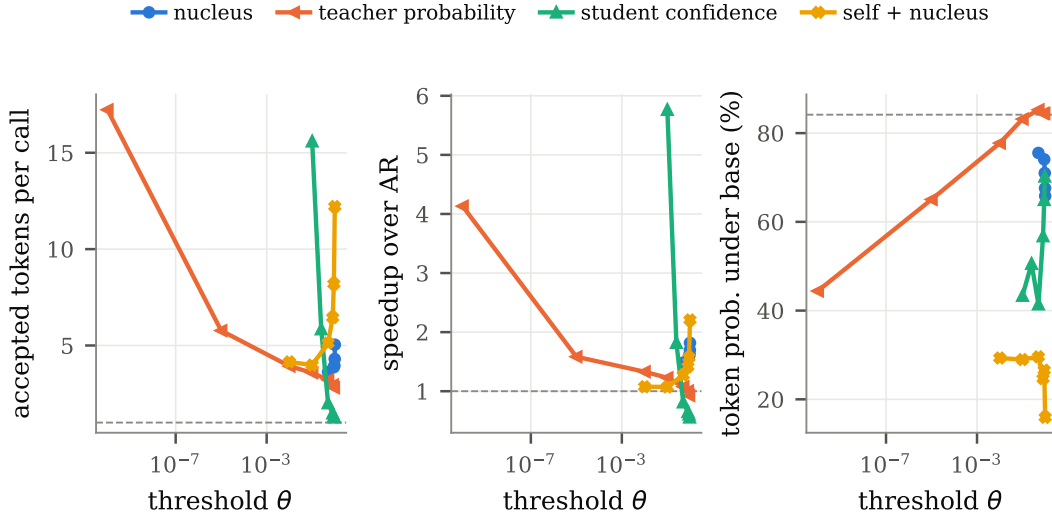


Figure 10: Threshold sweeps. Separate panels rather than twin axes, because the three measures are on unrelated scales. Dashed lines mark the autoregressive reference.

6.4 ENSEMBLES AND ALLOCATION

Table 4 varies how the per-strand acceptance probability driving the allocation of \$5 is estimated: a single pooled p , an online Bayesian update (conjugate Beta-Binomial, or a grid reweighted by the survival likelihood), and a small learned head predicting p from the hidden state. Differences are modest at $K = 5$ and grow at $K = 50$, consistent with the allocation mattering more when there is more budget to misallocate.

6.5 DOES THE MODEL PREDICT THE DATA?

Table 5 and Figure 5 validate §4. One three-parameter model, fitted jointly, reproduces the per-call histogram at every width with $R^2 \in [0.976, 0.996]$ and predicts $\mathbb{E}[G]$ to within 0.08 tokens at $K \leq 50$. It degrades at $K = 1000$ (predicting 5.09 against an observed 4.81), which is the

Table 4: Estimating the allocation’s acceptance probability. All rows use the nucleus criterion at the stated θ ; “allocation” is how p is estimated for the knapsack of §5. Preliminary.

verification	allocation	K	tok./call	ms/tok.	speedup	$\bar{q}$ (%)
verified against the base model, $\theta = 0.98$	uniform	5	4.32	57.4	$0.98 \times$	82.92
	Bayes, conjugate	5	4.46	55.9	$1.01 \times$	83.52
	Bayes, grid	5	4.38	48.2	$1.17 \times$	82.46
	uniform	50	4.74	140.4	$0.40 \times$	82.97
	Bayes, conjugate	50	4.77	159.8	$0.35 \times$	82.02
	Bayes, grid	50	4.60	182.1	$0.31 \times$	81.88
self-verified, $\theta = 0.8$	uniform	50	5.98	111.1	$0.51 \times$	73.74
	Bayes, conjugate	50	6.03	179.0	$0.32 \times$	76.04
	Bayes, grid	50	5.77	201.8	$0.28 \times$	74.29
	learned p -head	50	3.45	325.4	$0.17 \times$	74.48

Table 5: Model validation across ensemble widths. R^2 of the predicted per-call histogram for the pooled and heterogeneous models, and the likelihood-ratio statistic against a single pooled p (99 d.o.f.).

K	calls	$\bar{G}$ obs.	$E[G]$ model	$\hat{p}_{\text{pool}}$	R^2_{pool}	R^2_{het}	χ^2_{LRT}
1	5070	3.355	3.348	0.592	0.989	0.999	401
2	4630	3.626	3.563	0.563	0.996	0.999	431
5	4321	3.907	3.824	0.523	0.993	0.999	517
50	3870	4.489	4.412	0.449	0.976	0.996	478
1000	3581	4.807	5.085	0.365	0.980	0.988	742

extrapolation limit of a depth-uniform (π_0, p, ρ) . Integrating p over the fitted population equation 6 improves the fit at every width, by between +0.3 and +2.0 points of R^2 , but not monotonically in K : the largest gain is at $K = 50$ and the gain at $K = 1000$ is smaller than at $K = 1$ (Figure 11). Prompt-level heterogeneity is therefore real and worth modelling, but it is not the dominant source of the pooled model’s residual error, and the $K = 1000$ shortfall is not explained by it.

6.6 SELF-VERIFICATION

Dropping the base model entirely and verifying against the proposal model’s own autoregressive prediction is attractive — one set of weights, no second model — and it does raise tokens per call (3.86 exact, 6.45 with nucleus $\theta = 0.8$). But faithfulness collapses: $\bar{q} = 65.9$ for exact self-verification and 24.7 with nucleus. This is expected rather than surprising. Self-verification removes the only reference to the distribution we are asking the output to match, so the generation is free to drift, and the metric that detects drift duly does. Self-verification is the right tool when the goal is a fast standalone generator, and the wrong one when the goal is to accelerate a *specific* base model.

7 RELATED WORK

Speculative decoding proposes tokens with a cheap drafter and verifies them against the target model with a distribution-preserving acceptance rule (Leviathan et al., 2023; Chen et al., 2023); our exact rule is its analogue under shared auxiliaries. A large family of methods makes the drafter cheaper or better: extra decoding heads (Stern et al., 2018; Cai et al., 2024), sequentially dependent heads (Ankner et al., 2024), feature-space drafting with dynamic draft trees (Li et al., 2024a;b), Jacobi-style parallel decoding (Fu et al., 2024), and retrieval from the prompt (Saxena, 2023). Multi-token prediction has also been used as a training objective in its own right (Gloeckle et al., 2024). All of these inherit the acceptance rule rather than study it, and are benchmarked with accepted-tokens-per-call and speedup (Xia et al., 2024) but not with a test of distributional faithfulness. PTP (Draxler et al., 2025) supplies the proposal mechanism we build on, along with Partial Quadratic Decoding; the connection between auxiliary variables and inverse autoregressive flows (Kingma et al., 2016) is

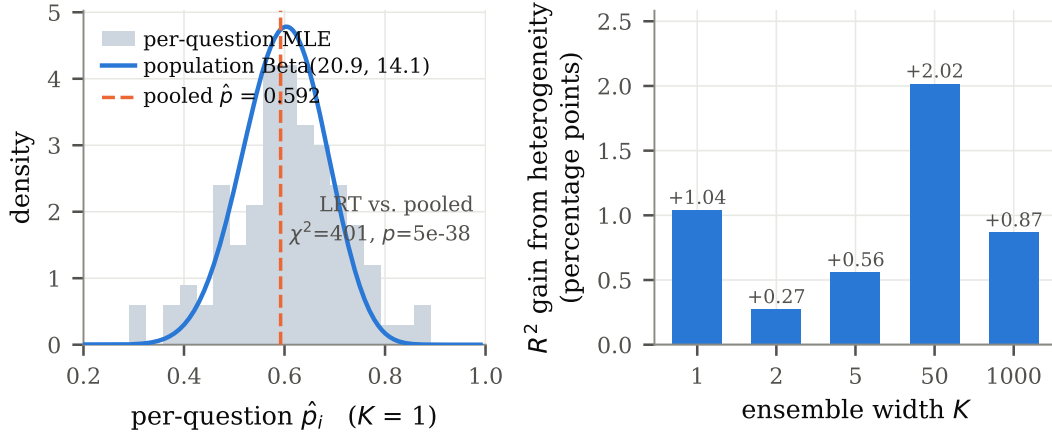


Figure 11: Prompt-level heterogeneity. **Left:** per-question acceptance probabilities at $K = 1$, the fitted population Beta(a, b), and the pooled estimate; the empirical spread is wider than the population because the per-question MLEs carry estimation noise. **Right:** the resulting gain in histogram fit, recomputed against the same predictions used in Figure 5 and Table 5. The gain is positive at every width but not monotone in K .

noted there. Our branching model is a zero-inflated, shared-frailty Galton–Watson process (Athreya & Ney, 1972).

8 CONCLUSION

Treating the acceptance rule as a design variable rather than a fixed detail changes the picture of multi-token inference. Accepted-tokens-per-call, wall-clock speedup, and faithfulness to the base model rank decoding rules differently, and reporting only the first — as is common — can recommend a configuration that is both slower and less faithful than the alternative. A three-parameter branching model of the number of correct tokens per call captures the per-call distribution across three orders of magnitude of proposal width, and makes the proposal-geometry question an exactly solvable knapsack whose solution also supplies the branch weights and branch values that Partial Quadratic Decoding needs.

Limitations. All numbers are preliminary, single-seed, and from one model pair (Vicuna-7B and one distilled student) on one GPU; the wall-clock conclusions in particular are hardware- and implementation-specific, and the ensemble rules would be read differently on hardware with spare parallel capacity. We evaluate only first turns of Spec-Bench, which leaves no reference completion, so our quality measures are all relative to the base model rather than to a ground truth; a rule that drifts towards *better* text than the base model’s would be penalized by p_{align} exactly as one that drifts towards worse. The fitted (π_0, p, ρ) is depth-uniform and checkpoint-specific, and its extrapolation degrades at $K = 1000$. The ratio-rule anomaly of §6.2 is unresolved.

REFERENCES

- Zachary Ankner, Rishab Parthasarathy, Aniruddha Nrusimha, Christopher Rinard, Jonathan Ragan-Kelley, and William Brandon. Hydra: Sequentially-dependent draft heads for medusa decoding. *arXiv preprint arXiv:2402.05109*, 2024.
- Krishna B. Athreya and Peter E. Ney. *Branching Processes*. Springer, 1972.
- Tianle Cai, Yuhong Li, Zhengyang Geng, Hongwu Peng, Jason D. Lee, Deming Chen, and Tri Dao. Medusa: Simple LLM inference acceleration framework with multiple decoding heads. In *International Conference on Machine Learning (ICML)*, 2024.

- Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. Accelerating large language model decoding with speculative sampling. *arXiv preprint arXiv:2302.01318*, 2023.
- Felix Draxler, Justus Will, Farrin Marouf Sofian, Theofanis Karaletsos, Sameer Singh, and Stephan Mandt. Parallel token prediction for language models. *arXiv preprint arXiv:2512.21323*, 2025.
- Yichao Fu, Peter Bai, Hao Zhang, and Ion Stoica. Break the sequential dependency of LLM inference using lookahead decoding. In *International Conference on Machine Learning (ICML)*, 2024.
- Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & faster large language models via multi-token prediction. In *International Conference on Machine Learning (ICML)*, 2024.
- Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. In *International Conference on Learning Representations (ICLR)*, 2020.
- Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
- Durk P. Kingma, Tim Salimans, Rafal Jozefowicz, Xi Chen, Ilya Sutskever, and Max Welling. Improved variational inference with inverse autoregressive flow. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.
- Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast inference from transformers via speculative decoding. In *International Conference on Machine Learning (ICML)*, 2023.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE: Speculative sampling requires rethinking feature uncertainty. In *International Conference on Machine Learning (ICML)*, 2024a.
- Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. EAGLE-2: Faster inference of language models with dynamic draft trees. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2024b.
- Apoorv Saxena. Prompt lookup decoding. <https://github.com/apoorvumang/prompt-lookup-decoding>, 2023.
- Mitchell Stern, Noam Shazeer, and Jakob Uszkoreit. Blockwise parallel decoding for deep autoregressive models. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
- Heming Xia, Zhe Yang, Qingxiu Dong, Peiyi Wang, Yongqi Li, Tao Ge, Tianyu Liu, Wenjie Li, and Zhifang Sui. Unlocking efficiency in large language model inference: A comprehensive survey of speculative decoding. In *Findings of the Association for Computational Linguistics (ACL)*, 2024.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

A ALL DECODING CONFIGURATIONS

Table 6 lists every configuration, sorted by wall-clock speedup. Re-run variants of identical configurations are excluded, as are runs marked superseded in the source tree. “gzip” is the compression ratio of the generated text, a model-free repetition proxy; lower means more repetitive.

Table 6: All 62 decoding configurations, sorted by speedup. Preliminary.

run	family	tok./call	ms/tok.	speedup	$\bar{q}$ (%)	PPL	gzip
ratio-p-0.5	ratio	15.20	5.8	9.70×	49.16	240.22	0.226
conf-p-0.1	student confidence	15.59	9.8	5.76×	43.36	124.70	0.125
thresh-0.0000000001	teacher threshold	17.23	13.6	4.13×	44.41	96.82	0.318
first_4	accept- k	4.00	16.1	3.50×	47.34	31.99	0.310
ratio-p-0.1	ratio	6.65	18.5	3.06×	45.10	19.04	0.520
ptp.self	self-verified	3.50	24.7	2.29×	66.21	1.61	0.550
seq-ptp-topp-self-0.99-tok	self-verified	12.21	25.6	2.21×	15.94	91.95	0.471
seq-ptp-self-aux	self-verified	6.45	29.0	1.94×	53.88	5.45	0.479
ratio-10000	ratio	2.61	30.9	1.83×	70.33	1.57	0.721
conf-p-0.25	student confidence	5.84	31.0	1.82×	50.57	57399.95	0.363
tmp.seq-ptp-topp-0.99	nucleus	5.04	31.0	1.82×	65.83	1.67	0.641
ratio-10	ratio	2.62	31.0	1.82×	70.12	1.57	0.717
first_2	accept- k	2.00	31.7	1.78×	49.29	40.92	0.515
ratio-2	ratio	2.49	32.3	1.75×	69.81	1.58	0.727
ptp	exact	2.39	32.7	1.73×	84.51	1.19	0.685
tmp.seq-ptp-topp-0.98	nucleus	4.30	33.2	1.70×	67.58	1.56	0.671
ratio-p-0.01	ratio	2.36	33.9	1.66×	62.33	1.83	0.736
ratio-p-0.001	ratio	2.35	34.1	1.66×	69.56	1.59	0.728
ratio	ratio	2.35	34.3	1.65×	69.94	1.58	0.726
tmp.seq-ptp-topp-0.95	nucleus	3.96	34.7	1.62×	70.99	1.46	0.687
ratio.g	ratio	2.31	34.9	1.62×	71.52	1.50	0.686
tmp.seq-ptp-topp-0.9	nucleus	3.88	35.4	1.59×	74.09	1.40	0.705
thresh-0.00001	teacher threshold	5.77	35.6	1.58×	65.05	1.71	0.588
seq-ptp-topp-self-0.9-tok	self-verified	8.20	35.9	1.57×	26.57	63.95	0.475
tmp.seq-ptp-topp-0.5	nucleus	3.62	38.0	1.48×	75.53	1.39	0.707
tmp.seq-ptp-topp-self-0.8-tok	self-verified	6.45	39.8	1.42×	24.66	38.70	0.537
first_l_5	accept- k	1.50	41.9	1.35×	35.54	18.19	0.742
thresh-0.01	teacher threshold	3.93	42.4	1.33×	77.75	1.31	0.671
tmp.seq-ptp-topp-self-0.5-tok	self-verified	5.12	44.4	1.27×	29.58	52.51	0.535
thresh-0.1	teacher threshold	3.58	46.0	1.23×	83.16	1.21	0.672
seq-ratio	ratio	3.43	48.1	1.17×	83.53	1.21	0.689
tmp.seq-ptp-topp-choice-5-0.98-bayes-p	nucleus + ensemble	4.38	48.2	1.17×	82.46	1.23	0.658
seq-ptp-self-tok	self-verified	3.86	49.9	1.13×	65.92	1.60	0.555
first_l_2	accept- k	1.20	51.6	1.09×	48.75	2.90	0.722
thresh-0.5	teacher threshold	3.22	52.3	1.08×	85.26	1.18	0.678
tmp.seq-ptp-topp-self-0.01-tok	self-verified	4.15	52.4	1.08×	29.31	21.01	0.569
tmp.seq-ptp-topp-self-0.1-tok	self-verified	3.98	52.6	1.07×	28.96	24.87	0.564
seq-ratio.g	ratio	3.21	53.0	1.06×	81.29	1.25	0.687
seq-ptp	exact	3.63	55.3	1.02×	84.71	1.19	0.687
tmp.seq-ptp-topp-choice-5-0.98-bayes-conjugate	nucleus + ensemble	4.46	55.9	1.01×	83.52	1.21	0.660
thresh-0.8	teacher threshold	2.95	56.1	1.00×	84.43	1.19	0.684
ar	autoregressive	1.00	56.4	1.00×	84.15	1.20	0.705
tmp.seq-ptp-topp-choice-5-0.98	nucleus + ensemble	4.32	57.4	0.98×	82.92	1.22	0.671
thresh-0.9	teacher threshold	2.81	60.8	0.93×	84.44	1.19	0.687
first_l	accept- k	1.00	61.4	0.92×	84.73	1.19	0.686
conf-p-0.5	student confidence	2.01	69.4	0.81×	41.42	17.03	0.512
conf-p-0.8	student confidence	1.48	86.8	0.65×	56.78	2.24	0.675
conf-p-0.9	student confidence	1.34	92.7	0.61×	64.95	1.77	0.679
conf-p-0.95	student confidence	1.27	101.1	0.56×	70.23	1.58	0.688
tmp.seq-ptp-topp-choice-50-self-0.8-tok	self-verified	5.98	111.1	0.51×	73.74	1.40	0.483
tmp.seq-ptp-topp-choice-50-self-0.5-tok	self-verified	5.70	116.6	0.48×	73.70	1.41	0.502
tmp.ptp-topp-choice-50-self-0.5	self-verified	3.10	129.3	0.44×	66.38	1.64	0.484
tmp.seq-ptp-topp-choice-50-0.98	nucleus + ensemble	4.74	140.4	0.40×	82.97	1.22	0.680
tmp.seq-ptp-topp-choice-50-self-0.98-tok	self-verified	6.07	145.2	0.39×	73.06	1.41	0.490
tmp.seq-ptp-topp-choice-50-0.98-bayes-conjugate	nucleus + ensemble	4.77	159.8	0.35×	82.02	1.23	0.685
tmp.seq-ptp-topp-choice-50-self-0.8-bayes-conjugate-tok	self-verified	6.03	179.0	0.32×	76.04	1.36	0.456
tmp.seq-ptp-topp-choice-50-0.98-bayes-p	nucleus + ensemble	4.60	182.1	0.31×	81.88	1.24	0.691
tmp.seq-ptp-topp-choice-50-self-0.8-bayes-p-tok	self-verified	5.77	201.8	0.28×	74.29	1.39	0.485
tmp.seq-ptp-topp-choice-50-self-0.8-phad-tok	self-verified	3.45	325.4	0.17×	74.48	1.39	0.512